

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260619_080211_779675 -->

Framework: JUnit | Run ID: run_20260619_080211_779675

Tóm tắt kỹ thuật

- Framework: JUnit 5 (JUnit Jupiter)
- Kết quả chạy: Tất cả 16 test đều pass, không lỗi biên dịch hay runtime.
- Coverage: 100 % (Jacoco) – đạt ngưỡng chất lượng “coverage”.
- Vấn đề thực thi: Không có lỗi thực thi; các test được biên dịch và chạy thành công.
- Trình tự kiểm tra: Kiểm thử được sinh cho BankAccount.java; các test liên quan tới các tính năng JUnit (TempDir, ParameterizedTest, migration) đã bị skip vì được đánh dấu *out_of_scope* trong test plan.

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- TC-001 – TC-005 (TempDir, ParameterizedTest, migration) được đánh dấu *out_of_scope* trong test plan, nhưng yêu cầu trong phần *Requirement JSON* vẫn yêu cầu kiểm thử các tính năng này (chia sẻ thư mục tạm, factory method/constructor, thay thế annotation JUnit 4).
- Bằng chứng: Trong test plan, các TC này có source_binding_status: out_of_scope và trong mã test có comment “Skipped TC-001 to TC-005 ... out_of_scope (JUnit framework features)”.
- Ảnh hưởng: Không có test nào xác nhận hành vi thực tế của @TempDir (chia sẻ thư mục), việc tạo đối tượng Book qua factory method, hay việc migration annotation. Điều này làm cho bộ kiểm thử không đáp ứng đầy đủ các yêu cầu đã nêu.
- Cách sửa: Thêm các test thực thi thực tế:
 1. TempDir: khai báo static @TempDir Path sharedTempDir; và viết hai test method ghi file test.txt vào cùng thư mục, sau đó đọc và xác nhận nội dung singletonList("a,b,c").
 2. ParameterizedTest: tạo lớp Book (hoặc mock) với factory method public static Book fromTitle(String) và viết @ParameterizedTest @ValueSource(strings = "42 Cats") để kiểm tra book.getTitle().
 3. Migration: viết test mẫu chuyển @RunWith(MockitoExtension.class) sang @ExtendWith(MockitoExtension.class) và xác nhận hành vi mock vẫn đúng; tương tự cho @Category → @Tag.
- TC-010 & TC-011 (clarification) không có test thực tế vì thiếu thông tin, nhưng chúng vẫn là các yêu cầu cần được làm rõ và có thể được chuyển thành test khi thông tin được cung cấp.

Assertion yếu

Không phát hiện.

Rủi ro maintainability

- Phụ thuộc vào trạng thái nội bộ qua helper `createInactiveAccount`: phương pháp này gọi `account.deactivate()` để tạo tài khoản không active. Nếu logic `deactivate()` thay đổi (ví dụ yêu cầu `balance = 0`), các test hiện tại có thể phá vỡ.
- Cách giảm rủi ro: Cung cấp một constructor hoặc factory method cho tài khoản không active (nếu có) hoặc mock `BankAccount` để thiết lập trạng thái `isActive = false` mà không phụ thuộc vào logic `deactivate()`.
- Hard-coded giá trị tolerance trong `assertEquals` cho double (0.001) được lặp lại nhiều lần; nếu thay đổi yêu cầu độ chính xác, cần cập nhật ở nhiều nơi.
- Cách giảm rủi ro: Định nghĩa một constant `private static final double EPS = 1e-3`; và dùng trong mọi `assertEquals` cho double.

Gợi ý sửa ngay

1. Thêm test cho `TempDir`, `ParameterizedTest` và migration (như mô tả ở mục `*Test` còn thiếu*) để đáp ứng đầy đủ yêu cầu.
2. Tách helper tạo tài khoản không active thành một phương thức tĩnh trong lớp test hoặc sử dụng một subclass mock để tránh phụ thuộc vào `deactivate()`.
3. Định nghĩa constant `EPS` cho các phép so sánh double để dễ bảo trì.
4. Kiểm tra lại coverage báo cáo: mặc dù hiện tại 100 %, hãy chắc chắn rằng các test mới (`TempDir`, `ParameterizedTest`) cũng được bao phủ.
